

New York University Tandon School of Engineering

Mechanical and Aerospace Engineering

ROB-GY 6013 Mathematics for Robotics

Problem Set 2: Machine Learning

Due Wednesday, October 21, 2026, before class (5:00 PM)

Submit on Gradescope.

Discussion with other students is allowed. Write your solutions alone.

Use any tool to understand the material, but not to write your solutions.

Solutions typeset in L^AT_EX or Markdown earn a 10% bonus. Show your reasoning: a bare final answer earns little credit.

Problem 1: Matrix calculus and least squares (15 points)

A linear model $\hat{y} = w^T x$ predicts a target y from an input $x \in \mathbb{R}^d$. Given n examples, with the inputs in the rows of $X \in \mathbb{R}^{n \times d}$ and the targets in $y \in \mathbb{R}^n$, the weights $w \in \mathbb{R}^d$ are chosen to minimize

$$L(w) = \frac{1}{2} \|Xw - y\|^2.$$

Your goal is to find the weights that minimize L . Use the data

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad y = \begin{bmatrix} 2 \\ 2 \\ 3 \end{bmatrix}.$$

- (a) (4 pts) Derive an expression for $\nabla L(w)$ in terms of X , y and w . You may use $\nabla_w(a^T w) = a$ and $\nabla_w(w^T A w) = 2Aw$ for a vector a and a symmetric matrix A .
- (b) (4 pts) Solve $\nabla L(w) = 0$ for w^* in terms of X and y , and state when this solution is unique. Then compute w^* for the data X and y above.
- (c) (4 pts) A common practice, called ℓ_2 regularization, is to add a penalty that discourages large weights and minimize

$$L_\lambda(w) = L(w) + \frac{\lambda}{2} \|w\|^2, \quad \lambda > 0,$$

instead. Find $\nabla L_\lambda(w)$ and solve $\nabla L_\lambda(w) = 0$ for w_λ^* in terms of X , y and λ .

- (d) (3 pts) Compute w_λ^* for the data X and y above with $\lambda = 1$, and compare it with w^* .

Problem 2: Gradient descent, step size, and weight decay (25 points)

Instead of solving $\nabla L(w) = 0$ directly, gradient descent starts from a guess w_0 and repeatedly updates

$$w_{t+1} = w_t - \eta \nabla L(w_t),$$

with a step size $\eta > 0$. For the loss and data of Problem 1, your goal is to find out where the iterates converge and for which step sizes they converge, through the following steps.

- (3 pts) Write one step of gradient descent in the form $w_{t+1} = Aw_t + b$. Express A and b in terms of X , y and η , then compute them for the data of Problem 1.
- (5 pts) Suppose we know that the recursion $w_{t+1} = Aw_t + b$ converges to some limit w_∞ as $t \rightarrow \infty$. What is w_∞ equal to? Express your answer in terms of A and b ; then, express the answer in terms of X , y and η . State any assumption you need. Finally, compute w_∞ for the data X and y of Problem 1.
- (4 pts) Let $e_t = w_t - w_\infty$ be the error after t steps. Derive a recursion for e_{t+1} in terms of e_t , and use it to express e_t in terms of e_0 .
- (4 pts) In (b) we assumed that the recursion $w_{t+1} = Aw_t + b$ converges to w_∞ . This happens exactly when $e_t \rightarrow 0$. Under what condition on A does $e_t = A^t e_0 \rightarrow 0$ for every e_0 ? Explain why.
- (5 pts) Let $\lambda_1, \dots, \lambda_d > 0$ be the eigenvalues of $X^\top X$, with eigenvectors $q_1, \dots, q_d$. Find the eigenvalues of A , and use (d) to find the range of η for which gradient descent converges. Then compute the eigenvalues of $X^\top X$ for the data and give the resulting range.
- (4 pts) Now run gradient descent on the penalized loss L_λ of Problem 1(c). Write one step in the form $w_{t+1} = c w_t - \eta \nabla L(w_t)$ and find c in terms of η and λ . Explain why this method is called *weight decay*.

Problem 3: Backpropagation through a two-layer network (20 points)

Consider a neural network with

- 2 input dimensions,
- 1 hidden layer with 2 hidden units and ReLU activation,
- 1 output dimension.

We train on a single example (x, y) , where $x \in \mathbb{R}^2$ is a column vector and $y \in \mathbb{R}$, with the loss $L = \frac{1}{2}(\hat{y} - y)^2$. The parameters are

- $W_1 \in \mathbb{R}^{2 \times 2}$ and $b_1 \in \mathbb{R}^2$: weights and biases from input to hidden layer,
- $W_2 \in \mathbb{R}^{1 \times 2}$ and $b_2 \in \mathbb{R}$: weights and bias from hidden layer to output.

Use the parameters and training example

$$x = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad y = 1, \quad W_1 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad W_2 = [2 \quad 1], \quad b_2 = -1.$$

- (a) (3 pts) Write down the complete mathematical formulas for the forward pass, calling the hidden pre-activations z , the hidden activations a and the output $\hat{y}$. Be explicit about each step. Use $\odot$ to denote element-wise multiplication.
- (b) (6 pts) Following the backpropagation algorithm, derive expressions for $\nabla_{\hat{y}}L$, $\nabla_{W_2}L$, $\nabla_{b_2}L$, ∇_aL , ∇_zL , $\nabla_{W_1}L$ and $\nabla_{b_1}L$, in this order, where ∇_vL is the gradient of L with respect to v , with the same shape as v . Each expression may use x , y , the forward-pass quantities z , a , $\hat{y}$, the parameters, and the gradients before it. Show how you apply the chain rule at each step.
- (c) (4 pts) For the values above, compute z , a , $\hat{y}$ and L . Then use your expressions from (b) to compute the numerical values of all the gradients in (b).
- (d) (3 pts) Take one gradient-descent step with step size $\eta = 0.05$: update every parameter θ to $\theta - \eta \nabla_{\theta}L$, using the gradients from (c). Compute a and L at the new parameters.
- (e) (4 pts) Redo (d) with step size $\eta = 0.1$, and compute a and L at the new parameters. Compare a and L with (d), and explain what happened. What happens if gradient descent continues from these parameters?

Problem 4: Computation graphs and automatic differentiation (20 points)

In Problem 3 you applied the chain rule by hand, one layer at a time. Libraries such as PyTorch do this automatically. During the forward pass they record every elementary operation, such as an addition or a multiplication, in a computation graph. During the backward pass they visit the graph in reverse order and apply the chain rule one operation at a time. In this problem you complete a small engine of this kind, which follows Andrej Karpathy's micrograd (optional background: <https://karpathy.ai/zero-to-hero.html>), and use it to check Problem 3 and to train a network.

Follow the starter notebook `ps2-autograd.ipynb` and complete the cells marked `TODO`. Do not use PyTorch or any other automatic-differentiation library. You do not submit the notebook: copy the code you wrote in every `TODO` cell into your write-up, and answer the questions below there.

- (a) (3 pts) Complete `_backward` for multiplication and for ReLU. Why does `_backward` use `+=` rather than `=`?
- (b) (5 pts) Write the forward pass of the network of Problem 3 with `Values` and call `backward()`. Give the printed gradients. Do they agree with your answer to Problem 3(c)?
- (c) (6 pts) Check every gradient of the random network in the notebook with centred finite differences. Report the number of parameters checked and the largest scaled error, which should be below 10^{-8} .
- (d) (6 pts) Train the network on samples of $\sin x$ with step size 0.1 for 1000 steps. Include the loss plot and the fit plot, and give the final loss, which should be below 0.01. Then train a fresh copy with step size 0.5 for 5 steps, and include its loss plot. What happens to the loss? Which result of Problem 2 does this match?